



Estamos por comenzar



Sesión 1

Nivelación

01

Introducción a Hardcore AI

¿Por qué existe el programa?

¿Cómo será la semana de nivelación?

Clases de nivelación: Lunes y miércoles de 7pm a 8:30pm

Sesiones de apoyo: Sesiones de tutoría con nuestro equipo técnico de apoyo: Jose Alanya y Cristian Córdoba

Miércoles 7pm - 8pm

Viernes 7pm - 8pm

Customer Success: Contamos con la plataforma Circle como centro de encuentro y repositorio de clases y materiales.
Andrea Caro

02

100 pesos de lo más básico

Inteligencia Artificial

02

Pero antes, veamos cómo estamos de conocimiento en IA...

Inteligencia Artificial (IA)

Campo de la informática dedicado a crear sistemas capaces de realizar tareas que normalmente requieren inteligencia humana (razonamiento, aprendizaje, percepción). Históricamente, se basaba en reglas estrictas programadas por humanos.

Ejemplo: Los motores de ajedrez antiguos o los NPCs básicos en videojuegos que siguen rutas predefinidas.

Machine Learning (ML)

Un subcampo de la IA donde ya no programamos las reglas explícitamente. En su lugar, le damos a la máquina grandes cantidades de datos y un algoritmo para que ella misma "aprenda" e infiera los patrones.

Ejemplo: Un sistema que analiza el histórico de logs de un servidor y predice cuándo va a fallar la memoria, o el filtro de spam de un correo electrónico.

Deep Learning (DL)

Una evolución del ML que utiliza Redes Neuronales Artificiales con múltiples "capas" (por eso es profundo). Estas redes imitan (matemáticamente) cómo el cerebro humano procesa la información, permitiendo manejar datos no estructurados como audio, imágenes o texto libre.

Ejemplo: Sistemas de reconocimiento facial (como FaceID) o traducción automática de idiomas (el motor detrás de Google Translate).

IA Generativa (Gen AI)

Es un subconjunto del Deep Learning impulsado por arquitecturas modernas (como los Transformers). Su objetivo no es analizar datos para tomar una decisión, sino aprender la estructura subyacente de esos datos para crear contenido nuevo: texto, imágenes, código, música.

Ejemplos: Claude, ChatGPT, Midjourney y GitHub Copilot.

04

Profundicemos en IA Generativa

Tokens · LLMs · Fine-Tuning · RAG · Embeddings · Agentes

Un token es una secuencia de caracteres que el modelo utiliza como su unidad básica de significado.

Proceso de Tokenización:

Paso previo donde el texto plano se convierte en IDs numéricos que la red neuronal puede procesar matemáticamente.

Eficiencia:

Permite que el modelo maneje prefijos, sufijos y raíces de palabras, dándole capacidad de "entender" idiomas y lenguajes de programación.

Reglas de cálculo:

- En inglés, 1,000 tokens $\approx$ 750 palabras.
- Aproximación rápida: 1 token $\approx$ 4 caracteres.
- Palabras comunes ("casa", "the") = 1 token.
- Palabras técnicas ("microservicios") = 2–3 tokens.
- Espacios e indentación en código también consumen tokens.

Aplicación Real:

Optimización de costos y aplicación de Rate Limit.

LLM (Large Language Models)

Un LLM es un motor estadístico de alta precisión. Su única tarea: *"Dado un texto de entrada, predecir cuál es el siguiente token más probable"*.

A diferencia del software basado en reglas, el LLM aprende patrones, estilos y relaciones lógicas durante su entrenamiento. No "sabe" cosas en el sentido humano — calcula probabilidades.

Large (Gran escala): Hace referencia tanto al tamaño del conjunto de datos de entrenamiento (Petabytes de texto) como al número de parámetros, que hoy oscilan entre 7 mil millones y 1.8 billones.

Language (Lenguaje): Su dominio principal es el lenguaje natural y formal (código), aunque hoy se extienden a otros modos como imágenes y audio.

Model (Modelo): Es una representación matemática probabilística de la realidad — no una base de datos de consulta.

Aplicación Real: Razonamiento lógico (seguir flujos de ejecución de código) · Traducción semántica (pasar un requerimiento en lenguaje natural a código funcional).

Fine-Tuning — Especialización de Modelos

El Fine-Tuning es el proceso de tomar un modelo ya pre-entrenado y someterlo a un entrenamiento adicional con un conjunto de datos pequeño, curado y específico.

Objetivo: Que el modelo aprenda nuevos **comportamientos, formatos o dominios** de conocimiento — no nuevos datos (para eso está RAG). A diferencia de un prompt, aquí sí se alteran los parámetros internos del modelo.

Dominio de Lenguaje Específico

Entrenar en terminología médica, legal o de ingeniería para que el modelo entienda y genere en ese vocabulario.

Estilo y Formato de Respuesta

El modelo aprende a devolver siempre JSON, XML o markdown con una estructura fija.

Tono de Marca

Adoptar la voz y el estilo de comunicación de una empresa específica.

Reducción de Latencia y Costos

Un modelo destilado y ajustado fino puede ser 10x más rápido y barato que el modelo general.

Aplicación Real: SQL Expert · Migración de Código entre lenguajes · Clasificación de alertas de Seguridad.

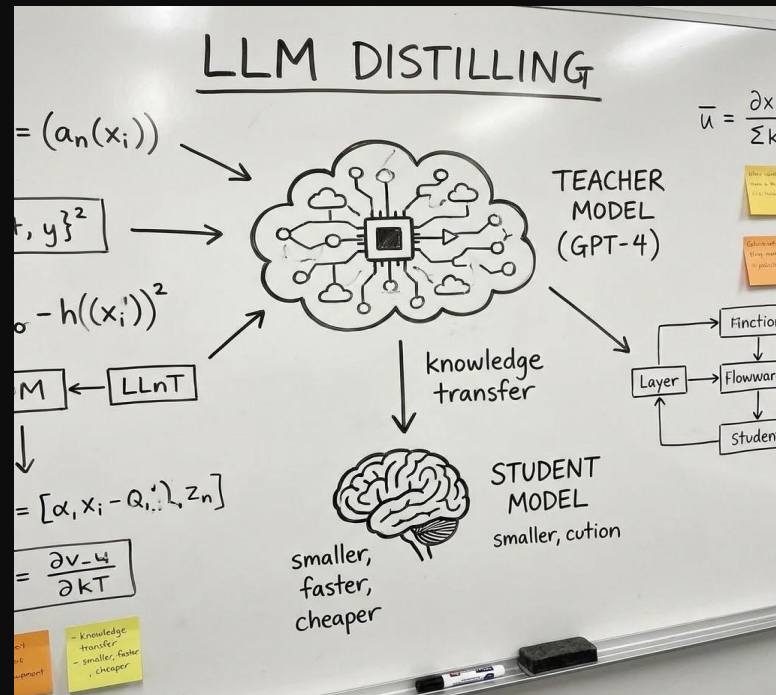
LLM Distilling: Destilación de Conocimiento

Proceso de transferir las capacidades de un modelo gigante y costoso (**Maestro**, ej. GPT-5) a un modelo pequeño y eficiente (**Estudiante**, ej. Llama 8B). El estudiante aprende a imitar el razonamiento del maestro.

Casos de uso:

- Copiloto de código que corre 100% offline en la laptop del desarrollador.
- Reducir costos de inferencia en un 90% usando un modelo destilado para clasificar tickets de soporte.

Aplicación Real: Ejecución de IA en dispositivos locales (Edge computing) o móviles para garantizar privacidad y latencia cero.

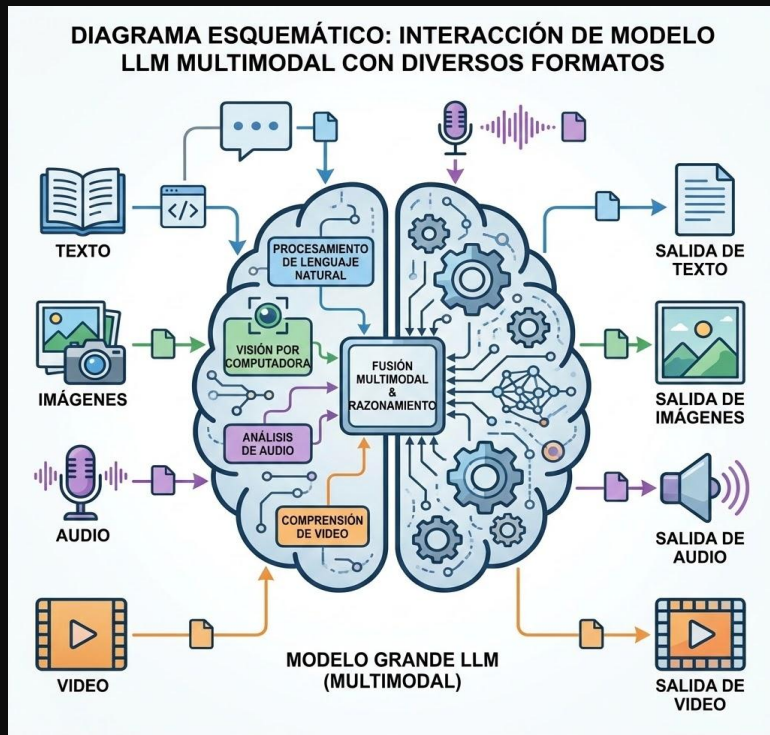


Modelos entrenados para procesar y relacionar múltiples tipos de datos (texto, imagen, audio y video) de forma nativa en un solo flujo de razonamiento.

Aplicaciones para desarrolladores:

- **Frontend-as-a-Service:** Subir una foto de un wireframe y obtener el código React/Tailwind.
- **Debugging Visual:** Cargar una captura de pantalla de un error de consola para que la IA explique la causa raíz.

Aplicación Real: Análisis de interfaces de usuario (UI/UX) y generación de código a partir de elementos visuales.



RAG (Retrieval-Augmented Generation)

Arquitectura que dota al LLM de una **"memoria externa"** de largo plazo. El conocimiento de los LLMs está congelado en su fecha de entrenamiento. RAG resuelve esto en tres pasos:

01

Retrieval (Recuperación)

Ante una pregunta, el sistema busca en documentos externos (PDFs, bases de datos, repositorios) la información relevante.

02

Augmentation (Aumentación)

Se combina la pregunta del usuario con la información encontrada, creando un "super-prompt" lleno de contexto.

03

Generation (Generación)

El LLM genera una respuesta basada únicamente en el contexto proporcionado, reduciendo casi a cero las alucinaciones.

Ejemplo: El usuario pregunta "¿Cómo escalamos el servicio de pagos?". El RAG busca en Confluence, encuentra el doc de "Arquitectura de Pagos v2" e inyecta ese contexto al LLM para que responda sin alucinar.

Aplicación Real: Base para asistentes que "conocen" tu código privado — responden sobre funciones específicas o lógica de negocio sin acceso público al repositorio.

Embeddings

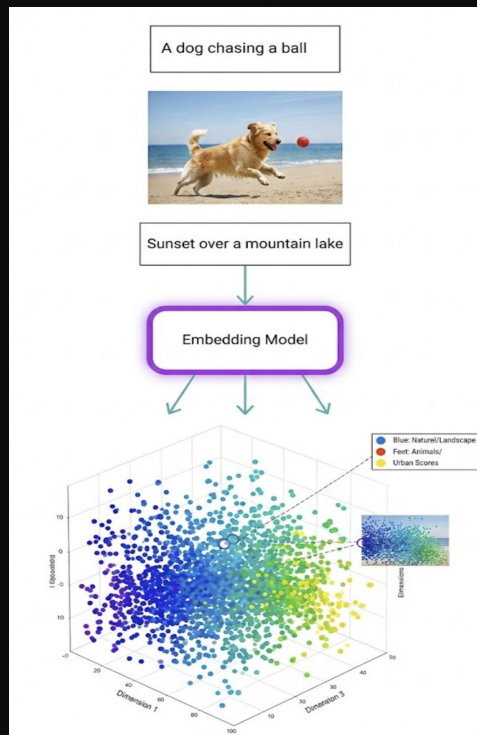
Vectores de números reales ([0.12, -0.5, 0.88...]) que traducen el lenguaje humano al lenguaje de las máquinas, posicionando palabras o frases en un espacio de cientos de dimensiones.

Propiedad Semántica: Textos con significados similares quedan "geográficamente" cerca en este mapa matemático.

Cálculo de Similitud: Se compara la distancia entre vectores (Similitud del Coseno) — no letras.

Ejemplo: Si buscas "problemas con el login", encuentras documentos sobre "error en la autenticación" aunque no comparten las mismas palabras.

Aplicación en Ingeniería: Es el motor de búsqueda que alimenta al RAG. Sin embeddings, la búsqueda es por palabras clave; con embeddings, la búsqueda es por concepto.



Bases de Datos Vectoriales: El Almacén del Contexto

A diferencia de las bases de datos relacionales (valores exactos) o de texto (palabras clave), las **Vector DBs** están diseñadas para almacenar embeddings y ejecutar **Similarity Search**: encontrar los N elementos más cercanos a una consulta en milisegundos.

Pinecone · SaaS · Líder en velocidad y escala

Totalmente administrado, sin gestionar infraestructura. Ideal para producción.

Weaviate / Qdrant · Open Source · soberanía de datos

Despliegue local o en nube. Popular en entornos con requisitos de privacidad.

ChromaDB · Prototipado rápido · in-memory

La favorita para desarrollo. Se ejecuta en un script de Python sin infraestructura.

pgvector · Extensión de PostgreSQL

Añade búsqueda vectorial si ya usas Postgres. Sin infraestructura adicional.

Milvus · Escala masiva · corporaciones

Diseñada para billones de vectores. Usada en grandes empresas.



Estamos por comenzar

05

Agentic AI

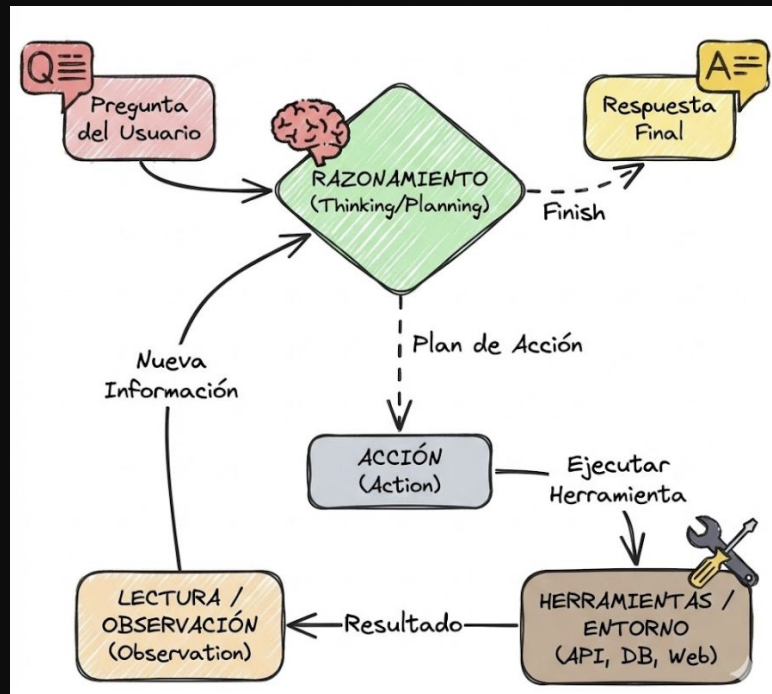
¿Qué es un Agente?

Un agente es un sistema de software que utiliza un LLM como su **unidad central de razonamiento** para alcanzar una meta. A diferencia de un chatbot que solo responde, un agente tiene un ciclo de vida:

Observa → Piensa → Actúa

- El agente no solo "sabe", sino que "hace".
- Se le otorgan **Herramientas (Tools)**: funciones de código (leer un archivo, ejecutar SQL, llamar a la API de Stripe) que él decide cuándo y cómo invocar.

Aplicación Real: "Encuentra el bug en el módulo de pagos". El agente lee los logs, busca el código relevante, escribe un test que falle para confirmar el bug, y propone el fix.



Arquitecturas de Agentes: El Cerebro Operativo

Para que un agente no se pierda en un bucle infinito, necesita una estructura de pensamiento. Las 3 más relevantes hoy:

01

ReAct (Reason + Act)

El estándar de oro. El agente escribe un Pensamiento (qué cree que debe hacer), ejecuta una Acción (llama a una herramienta) y recibe una Observación (el resultado). Repite hasta cumplir la meta.

Mejor para: tareas cortas e iterativas

02

Plan-and-Execute

El agente primero desglosa la meta en una lista de pasos (el plan) y luego los ejecuta secuencialmente. Mucho más eficiente para tareas largas donde cada paso es costoso.

Mejor para: tareas largas y estructuradas

03

Self-Reflection

Arquitectura de "doble chequeo". Un agente genera una solución y un segundo paso de crítica analiza el resultado en busca de errores, obligando al agente a iterar antes de entregar la respuesta.

Mejor para: alta calidad, baja tolerancia a errores

Ventana de Contexto (Context Window)

La ventana de contexto es la cantidad máxima de información (en tokens) que un modelo puede procesar, recordar y considerar en una sola interacción. **Todo lo que esté fuera de esta ventana, para el modelo no existe.**

Input + Output: El límite incluye tanto lo que envías (prompt, historial, adjuntos) como la respuesta que el modelo va a generar.

Evolución: Hace dos años el límite era de 4,000 tokens (~10 páginas). Hoy los modelos top manejan más de 1 millón de tokens — cientos de miles de líneas de código o libros enteros.

La trampa del contexto largo: Que un modelo pueda recibir 1M de tokens no significa que los atienda por igual. Los modelos recuerdan bien el principio y el final del contexto, pero tienden a "olvidar" la información en el medio.

RAG vs. Long Context: Si el repo es pequeño → mételo todo en la ventana. Si es grande → usa RAG para inyectar solo los 3 archivos relevantes.

Manejo de Errores: Cuando el IDE empieza a proponer código que rompe variables ya definidas, la ventana de contexto se saturó. Solución: limpiar el chat y empezar uno nuevo.

Workflows Agénticos (Agentic Workflows)

Un workflow agéntico es el diseño de una **"cadena de montaje"** para la IA. En lugar de un prompt masivo esperando un milagro (Zero-shot), se divide el problema en pasos iterativos donde la salida de uno es la entrada del siguiente.

Reflexión

La IA genera código, otra instancia lo revisa y critica, la primera lo corrige antes de mostrar la salida final.

Uso de Herramientas

El flujo se pausa para que un script ejecute el código generado, capture el error y se lo devuelva a la IA para que lo arregle.

Planificación

Antes de escribir una sola línea, la IA genera un documento de diseño paso a paso que el humano aprueba.

Ejemplo de workflow en un Pull Request:

1. Lee los commits y genera un resumen en lenguaje natural.
2. Revisa si hay vulnerabilidades de seguridad (OWASP Top 10).
3. Comenta directamente en la línea de código de GitHub sugiriendo el fix si encuentra un error.

Aplicación Real: Cada vez que se abre un Pull Request, el workflow anterior se dispara automáticamente.

Equipos de Agentes

Arquitectura avanzada donde múltiples agentes de IA — cada uno con un rol especializado, prompt de sistema diferente y herramientas específicas — colaboran entre sí para resolver un requerimiento complejo. **La especialización aumenta exponencialmente la calidad del output.**

Ejemplo: crear un nuevo endpoint (CrewAI / AutoGen)

Agente Tech Lead

Recibe el ticket, analiza el impacto y divide la tarea en sub-tareas especializadas.

Agente DBA

Diseña el modelo relacional, crea las tablas y genera los scripts SQL optimizados.

Agente Backend Java

Toma las especificaciones del DBA y programa los DTOs, entidades y el controlador REST en Springboot.

Agente QA

Revisa el código para detectar inyección SQL, valida códigos HTTP. Si algo falla, se lo devuelve al agente Backend.

Frameworks: CrewAI · Microsoft AutoGen · LangGraph · Swarm

**Es
todo
por
hoy**

